



**UFR Sciences
et Techniques**

TROISIÈME ANNÉE LICENCE INFORMATIQUE

RAPPORT DE PROJET

Projet Diagrammes UML

Lydia AIT SAHEL

Mazarine DAHMANI

Yanis SADOON

Rayane ARACHE

Nassim BENCHIKH

Sous la direction de :

M. Stéphane HERAUVILLE

Année Universitaire

2023/2024

Remerciements

Nous souhaitons exprimer nos plus sincères remerciements aux personnes qui nous ont apporté leur aide pour mener à bien ce projet. Au départ, ce projet nous semblait difficile, mais notre professeur s'est assuré que nous disposions de tout ce dont nous avons besoin.

Nous tenons également à exprimer notre gratitude envers les membres du groupe qui ont précédemment travaillé sur ce projet et ont rendu leur code disponible, ce qui nous a été d'une grande aide. Nous tenons à remercier chaleureusement M. Stéphane HERAUVILLE, notre professeur encadrant, pour son écoute attentive et sa disponibilité tout au long de la réalisation de notre projet. Nous lui sommes profondément reconnaissants pour l'aide inestimable qu'il nous a apportée et le temps précieux qu'il nous a généreusement consacré, sans son soutien ce projet n'aurait pu voir le jour.

En outre, nous tenons à exprimer notre sincère gratitude envers nos collègues de groupe, dont la contribution et la collaboration ont été essentielles à la réussite de ce projet. Leur engagement et leur travail acharné ont grandement contribué à atteindre les objectifs fixés. Enfin, nous souhaitons également exprimer nos plus sincères remerciements à nos parents respectifs pour leur soutien inébranlable et leur encouragement constant tout au long de cette année.

Table des matières

I	Introduction	4
II	Présentation du projet	5
II.1	Etude de l'existant	5
II.2	Architecture statique	6
II.3	Objectifs du projet	6
III	Organisation du code	7
IV	Notre solution	9
IV.1	Organisation du travail	9
IV.2	Technologies utilisées	9
IV.2.1	Configurations requises	9
IV.2.2	Java/JavaFX	9
IV.2.3	Outils techniques	9
IV.2.4	Outils collaboratifs	10
IV.3	Travail réalisé	10
IV.3.1	Mises à jour des dépendances	10
IV.3.2	Analyse du code	10
IV.3.3	Correction de la méthode de construction d'une relation	12
IV.3.4	Positionnement manuel des relations	12
IV.3.5	Gestion des superpositions	13
IV.3.6	Exportation PDF	13
V	Difficultés rencontrées	14
VI	Evolution potentielles	15
VII	Conclusion	16

Table des figures

1	Architecture statique.	6
2	Paquetages principaux de l'application.	7
3	Paquetage diagram.	7
4	Paquetage view.	8
5	Paquetage component.	8
6	Répartition des tâches.	9
7	Répartition des tâches.	11
8	Répartition des tâches.	12

I Introduction

Actuellement en troisième année de licence informatique à l'Université de Rouen, nous sommes engagés dans un projet de fin d'études sous la supervision de M. Stéphane HERAUVILLE. Notre tâche consiste à améliorer une application de création de diagrammes UML. Nous avons constaté, ainsi que nos enseignants, que malgré l'importance des diagrammes UML, les outils disponibles sont soit incomplets, soit trop coûteux, surtout pour des utilisateurs au budget limité.

Dans ce rapport, nous présenterons notre contribution au développement de cette application, en mettant particulièrement l'accent sur l'amélioration des fonctionnalités liées aux diagrammes de classe. Nous détaillerons les technologies et les approches techniques que nous avons adoptées, ainsi que les objectifs que nous nous sommes fixés pour ce projet et nos réalisations. De plus, nous aborderons les pratiques de gestion de projet que nous avons mises en œuvre.

Ce projet nous offre l'opportunité de mettre en pratique nos connaissances théoriques tout en contribuant à la création d'un outil qui sera bénéfique pour les futurs étudiants.

II Présentation du projet

II.1 Etude de l'existant

L'application existante "UML Reverse" est destinée à la création, la gestion et la manipulation des diagrammes UML. Développée initialement avec Java 7, l'application utilise Maven comme outil de gestion des dépendances et l'intégration continue quant à l'interface utilisateur cette dernière a été réalisée avec JavaFX.

- Points forts :

- Fonctionnalités de base : les fonctionnalités essentielles de l'application, telles que la création, la sauvegarde, et l'ouverture de projets sont fonctionnelles.
- Création et gestion de diagrammes : l'application permet efficacement la création de divers types de diagrammes UML, ainsi que des types, attributs, méthodes, et paramètres associés à ces diagrammes.
- Exportation : elle offre des options d'exportation en formats PlantUML et PNG.
- Importation : supporte l'importation à partir de fichiers PlantUML et de paquets JAVA.

- Points faibles :

- Interface utilisateur : il existe plusieurs problèmes dans l'interface, comme la superposition des éléments, où les relations se superposent sur les entités ainsi que le manque de contrôle manuel sur la position des éléments, l'application ne permet pas à l'utilisateur d'ajouter ou de supprimer des points de contrôle sur les relations.
- Absence d'exportation en PDF : l'application ne propose pas d'option pour exporter les diagrammes en format PDF.
- Gestion de multiples projets : il est impossible de gérer plusieurs projets simultanément.
- Importation de classes Java : ne permet pas l'importation directe de classes Java, ce qui pourrait faciliter l'intégration avec des projets existants.
- Dépendances obsolètes.

II.2 Architecture statique

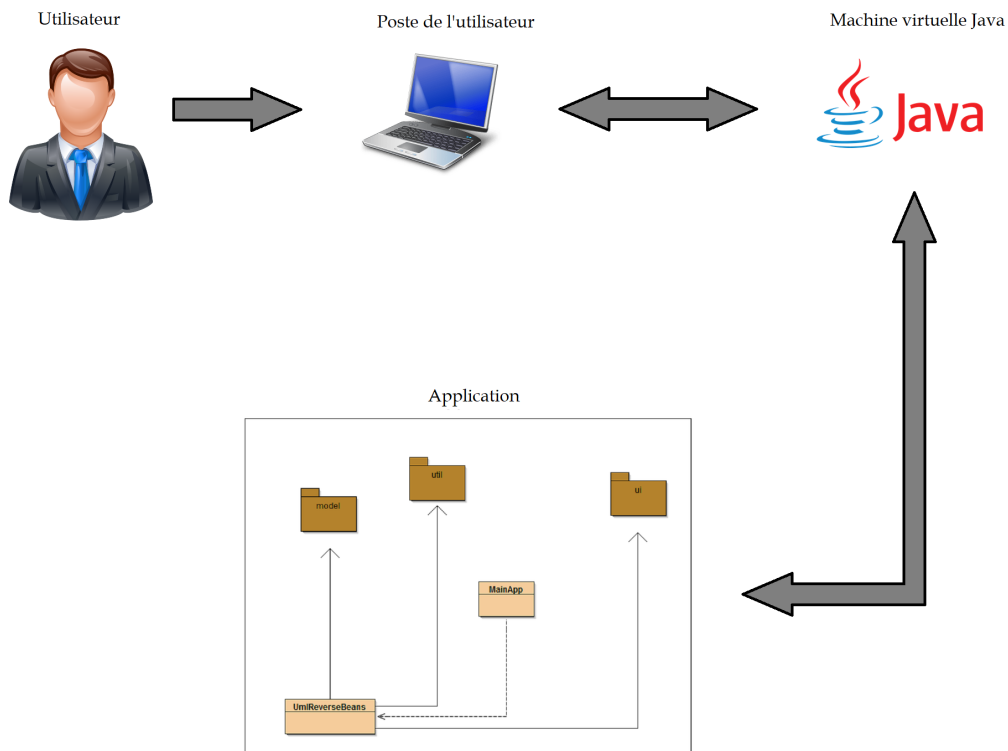


Figure 1: Architecture statique.

II.3 Objectifs du projet

Notre projet a pour objectif d'améliorer l'application existante, dont la première phase du projet a été développée en 2016 et la seconde en 2017 par des étudiants en Master 1 GIL. Ces objectifs sont définis comme suit :

1. **Conformité aux normes UML** : respect des spécifications UML pour le diagramme de classe.
2. **Gestion de projets UML** : création, sauvegarde, ouverture et gestion de plusieurs projets UML de manière ergonomique.
3. **Manipulation graphique des classes** : création, modification et suppression fluide et intuitive des classes.
4. **Positionnement manuel des relations entre les classes** : possibilité de déplacer manuellement les points de connexion sur les liens. Maintien des liens attachés aux classes même en cas de déplacement de ces dernières. Affichage clair des relations indépendamment des manipulations effectuées.

III Organisation du code

Le code de l'application UML Reverse est structuré en plusieurs paquets :

- Paquetage **Model** : ce paquetage représente les entités de l'application. Il est répartie en trois sous-paquetages **diagram**, **io** et **project**.
- Paquetage **ui** : représente la partie vue et controleur de l'application, ce paquetage est divisé en deux sous-paquetages **view** et **component**.
- Paquetage **util** : ce paquetage contient des classes utilitaires utilisées à travers l'application pour les testes des conditions et la gestion des erreurs.

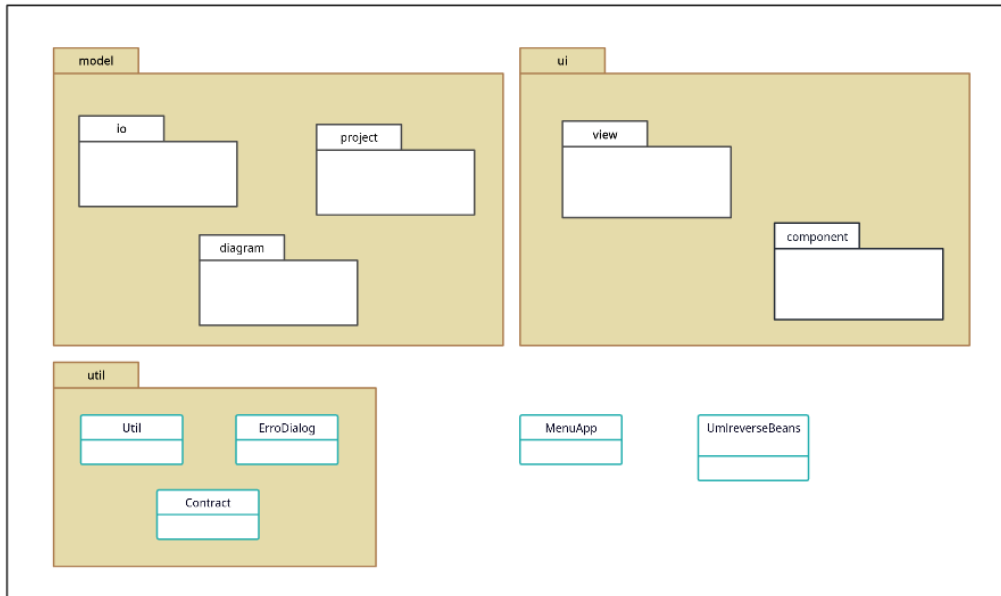


Figure 2: Paquetages principaux de l'application.

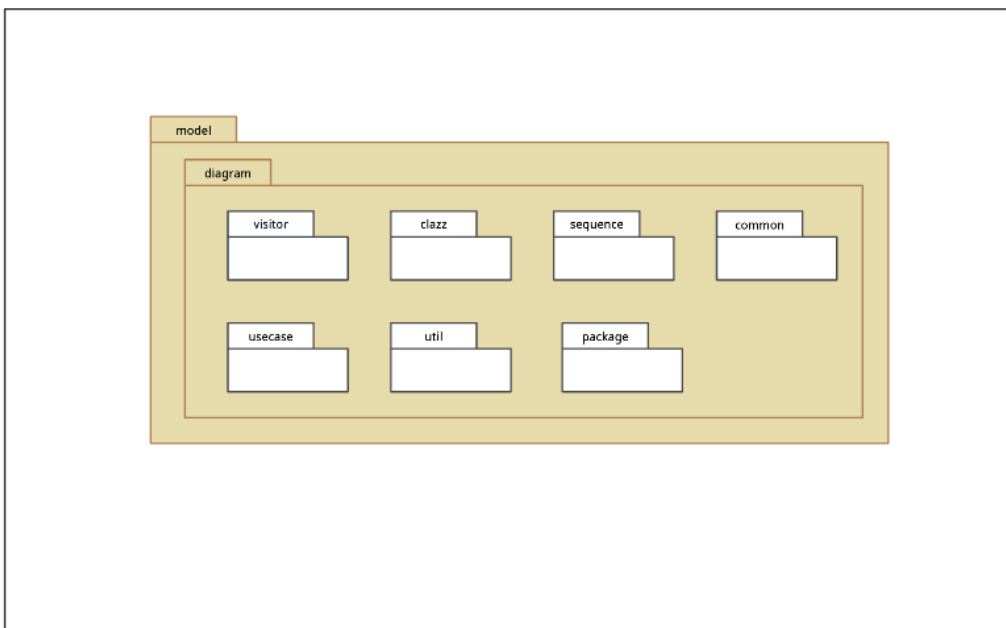


Figure 3: Paquetage diagram.

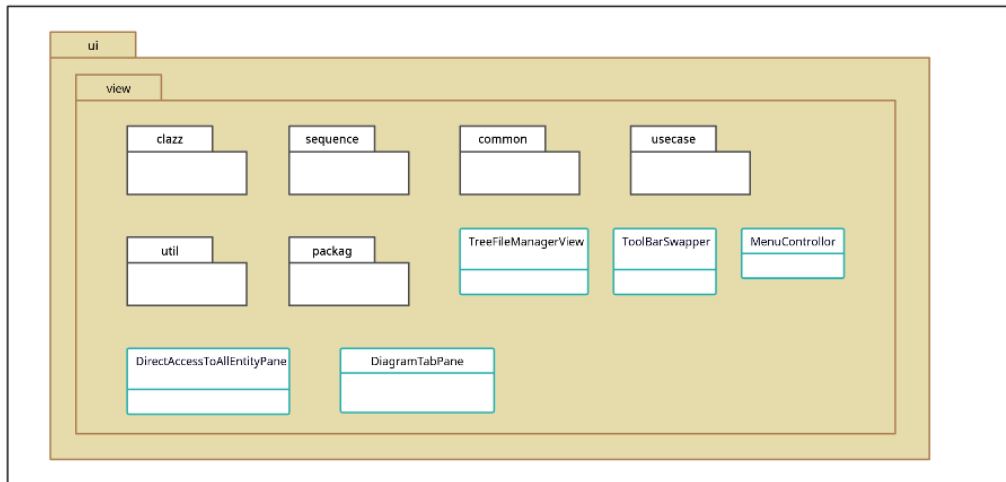


Figure 4: Paquetage view.

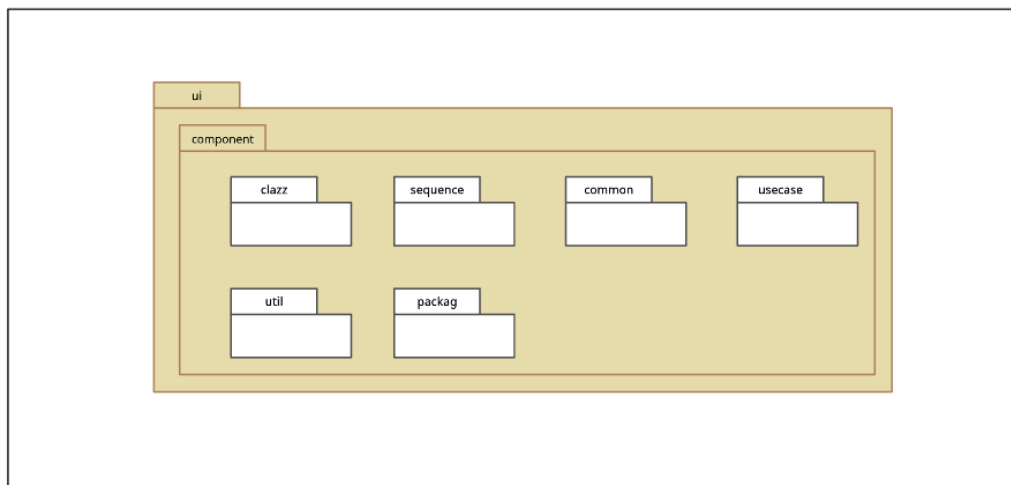


Figure 5: Paquetage component.

IV Notre solution

IV.1 Organisation du travail

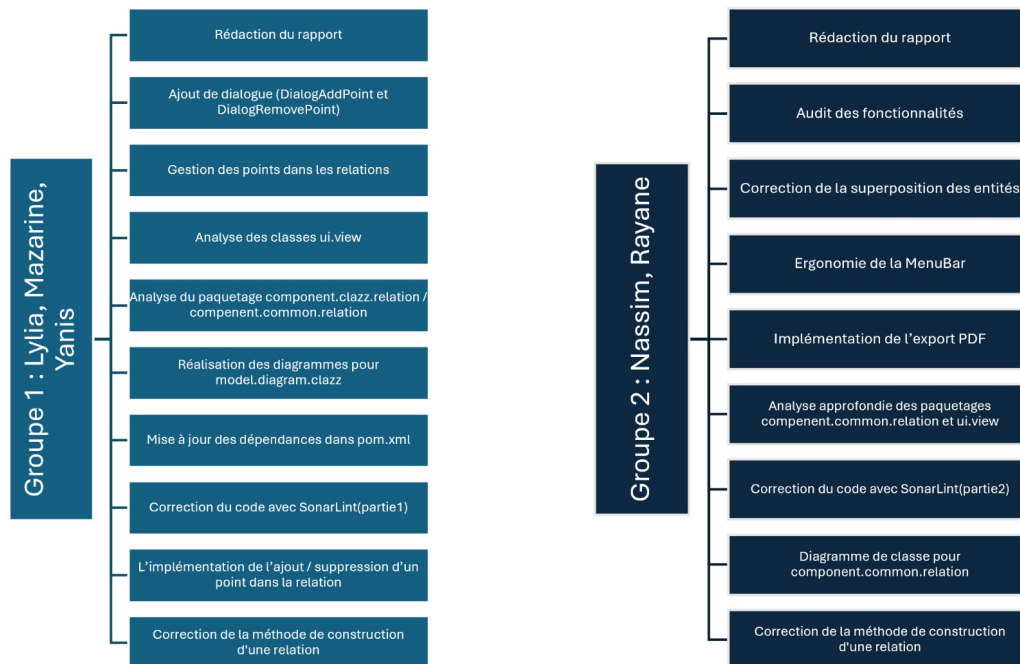


Figure 6: Répartition des tâches.

IV.2 Technologies utilisées

IV.2.1 Configurations requises

Avant de débiter notre projet, il était primordial de mettre en place un environnement de développement approprié afin de garantir une fondation solide pour notre travail. Nous avons donc procédé à l'installation des dernières versions TLS des outils nécessaires. Pour le langage de programmation, nous avons opté pour l'installation d'**openJDK 17**, tandis que pour la gestion de l'interface utilisateur **JavaFX 21.0.1** a été installé et **Maven 3.6.3** comme outil de gestion et d'automatisation.

IV.2.2 Java/JavaFX

- **Java** : est un langage de programmation de haut niveau populaire et polyvalent, largement utilisé dans le développement logiciel. Java utilise le concept de la programmation orienté objet, qui est un paradigme de programmation qui repose sur le concept d'objets, des entités qui peuvent contenir à la fois des données et des procédures. Java est un langage portable, c'est à dire que les programmes Java peuvent être exécutés sur diverses plates-formes informatiques sans nécessiter de modifications majeures.
- **JavaFX** : est un cadriciel et une bibliothèque complète et moderne pour écrire des applications en Java avec une interface graphique. Elle offre un ensemble complet d'outils pour la création d'interfaces utilisateur modernes, notamment des composants graphiques, des effets visuels, des animations et des styles. Il prend en charge l'utilisation de feuilles de style en cascade (CSS) pour la conception des interfaces utilisateur, ce qui permet une personnalisation facile de l'apparence des composants graphiques.

IV.2.3 Outils techniques

- **Eclipse** : est un environnement de développement intégré source ouvert qui possède une architecture modulaire extensible, et qui fournit des fonctionnalités complètes pour éditer, compiler, déboguer et déployer des applications logicielles.

- **SonarLint** : extension qui permet de détecter des bugs relatifs aux vulnérabilités du code et propose des solutions pour les ajuster. Les erreurs de code sont détectées au même moment où l'on code, à l'instar des vérificateurs d'écriture de Word (spell-checker). SonarLint pointe directement où l'erreur se situe et les trie en 3 couleurs :
 - Couleur bleu : suppression, renommage, complément, etc.
 - Couleur rouge : ajout, définition de constantes, problèmes de complexité, etc.
 - Couleur orange : changements (par exemple le changement de visibilité), propositions d'améliorations, etc.
- **Scenebuilder** : est un outil de conception graphique créé par Oracle et utilisé pour créer des interfaces utilisateur pour les applications JavaFX.
- **Maven** : est un outil qui aide les développeurs à gérer et à construire leurs projets Java de manière plus efficace en automatisant plusieurs tâches, comme la gestion des dépendances et la compilation du code.

IV.2.4 Outils collaboratifs

- **GoogleDoc** : application de traitement de texte en ligne de la suite Google, qui permet la rédaction, l'édition et le partage de documents.
- **GoogleMeet** : plateforme de vidéoconférence de la suite Google, qui permet de réaliser des réunions vidéos en ligne.
- **WhatsApp** : application de messagerie instantané que nous avons utilisé pour nos échanges au quotidien.
- **Overleaf** : plateforme en ligne gratuite qui permet de rédiger du texte en $\text{\LaTeX}$ de manière collaborative.
- **GitHub** : plateforme en ligne qui facilite la collaboration sur des projets de développement de logiciels en fournissant des outils pour héberger, partager, suivre les modifications et gérer les versions du code source.

IV.3 Travail réalisé

IV.3.1 Mises à jour des dépendances

L'une des premières étapes du travail réalisé a été de mettre à jour différentes dépendances utilisées pour améliorer la qualité du développement logiciel. Les dépendances mises à jour sont les suivantes :

Dépendance	Description	Version
JUnit	Framework de test unitaire, contient des TestCase qui sont des classes contenant des méthodes de tests.	4.12
Mockito	Framework pour le mock* d'objets en tests unitaires. * Fait référence à des objets qui simulent le comportement d'autres objets réels dans un test.	5.10.0
Commons-lang3	Fournit une bibliothèque d'utilitaires helper pour travailler avec les classes fondamentales du langage Java.	3.14.0
Assertj	Bibliothèque Java qui fournit des assertions pour améliorer les tests unitaires.	3.25.3
antlr4	Générateur d'analyseurs pour lire, traiter, exécuter ou traduire du texte structuré ou des fichiers binaires.	4.13.1

Table 1: Tableau des mises à jour des dépendances

IV.3.2 Analyse du code

Après avoir assuré un environnement adéquat pour réaliser le projet, nous avons commencé par l'analyse du code hérité, en nous concentrant spécifiquement sur les paquetages `ui.view` et `ui.component`, qui sont cruciaux pour l'interface utilisateur de notre application. Cette analyse a impliqué plusieurs classes clés.

- `fr.univrouen.umlreverse.ui.view` :

- La classe `MenuController` gère les interactions utilisateur avec le menu de l'application. Elle est responsable de traiter les événements tels que l'ouverture de projets, l'importation et l'exportation de fichiers, l'impression, etc.
- `TreeFileManagerView` étend `TreeView` de JavaFX pour afficher des données hiérarchiques telles qu'une structure de fichiers ou de projet. Elle maintient des références aux données de l'application et utilise des cartes pour associer différents types de diagrammes à leurs représentations dans l'arbre. Un `ChangeListener` est présent pour mettre à jour la vue lorsque le projet change.

- `fr.univrouen.umlreverse.ui.view.clazz` : la classe `ClassController` est un contrôleur dans le modèle MVC pour un diagramme de classes UML. Elle maintient une référence à un `IClassDiagram`, gérant ainsi les éléments du diagramme de classes. De plus, elle importe des classes liées aux interactions utilisateur, telles que des dialogues et des éléments d'interface utilisateur, ainsi que des classes de JavaFX pour gérer les menus contextuels.

- `fr.univrouen.umlreverse` : `MainApp` est le point d'entrée de l'application JavaFX. Elle étend `Application`, qui est une classe de base pour toutes les applications JavaFX. La classe `UmlReverseBeans` est responsable de la gestion des propriétés partagées à travers l'application et de la notification des changements de ces propriétés aux parties intéressées de l'application. Ces propriétés sont observables, ce qui signifie que d'autres parties de l'application peuvent être notifiées lorsque ces propriétés changent.

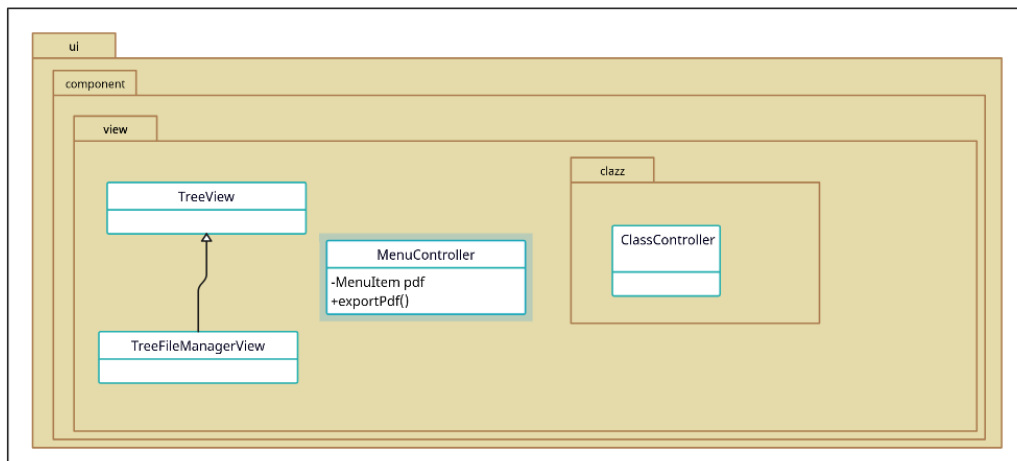


Figure 7: Répartition des tâches.

- `fr.univrouen.umlreverse.ui.component.common.relation` :

- `ArrowBody` manipule le corps d'une flèche dans un diagramme UML. Elle permet l'ajout et la suppression de points, ce qui ajuste la forme et la trajectoire de la flèche pour personnaliser les relations entre les éléments du diagramme.
- `IArrowBody` est une interface pour la représentation graphique du corps d'une flèche dans un diagramme UML. Cette interface définit plusieurs méthodes pour manipuler et accéder aux composants graphiques de la flèche.
- `ArrowHead` est une implémentation d'une tête de flèche dans un diagramme UML. Elle fournit des méthodes pour obtenir et définir les propriétés de la tête de flèche, y compris sa position, son type, son côté et le texte associé.
- `IArrowHead` définit le contrat pour une représentation graphique de la tête d'une flèche dans un diagramme UML. Elle fournit des méthodes pour obtenir les composants graphiques de la tête de la flèche.
- `ARelationGraphic` fournit une base pour représenter une relation dans un diagramme UML, y compris la flèche de la relation, les points d'extrémité et le texte associé.
- `IRelationGraphic` définit les opérations qui peuvent être effectuées sur une relation dans un diagramme UML, y compris l'accès aux composants graphiques de la relation et l'obtention de la cardinalité de fin.

- **IRelationGraphicController** définit les opérations qui peuvent être effectuées sur un contrôleur de relation dans un diagramme UML, y compris l'accès aux propriétés de la relation.
- **RelationGraphicController** est une implémentation de l'interface **IRelationGraphicController**, elle fournit une implémentation pour gérer une relation graphique dans un diagramme UML, y compris le contrôle des événements de déplacement et la gestion du menu contextuel.

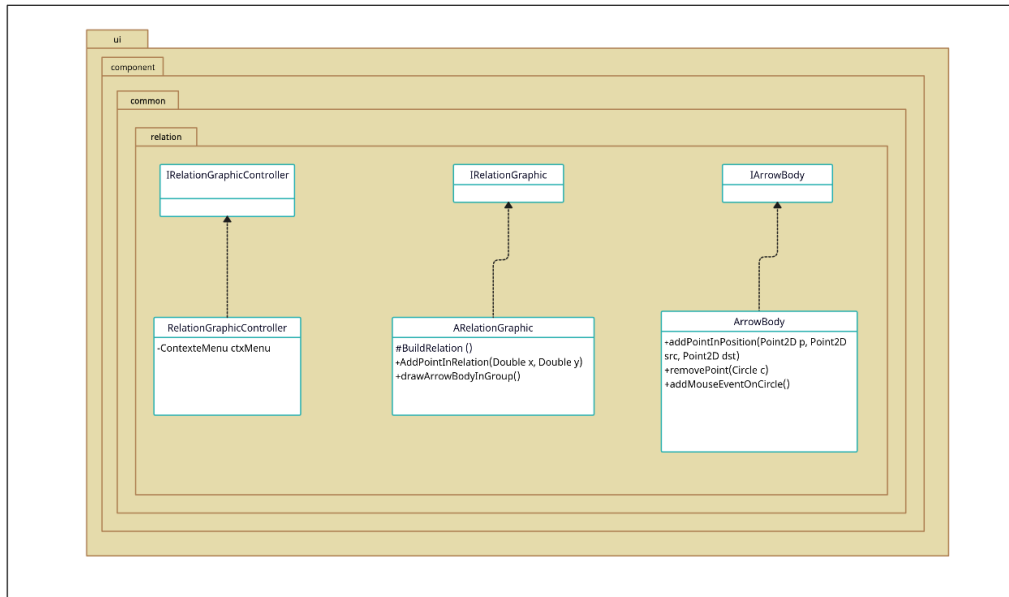


Figure 8: Répartition des tâches.

IV.3.3 Correction de la méthode de construction d'une relation

Pour simplifier la création des relations entre les entités de nos diagrammes, nous avons remplacé les deux anciennes méthodes par une seule, appelée **buildRelation()**. Cette nouvelle méthode produit une ligne droite agrémentée de trois points stratégiques : les extrémités attachées aux entités source et destination, et un point central qui introduit une flexibilité dans la représentation de la relation.

Pour gérer les différentes orientations possibles — verticale, horizontale ou diagonale — la méthode intègre des conditions qui ajustent automatiquement la position de la flèche en fonction de la localisation des entités concernées:

La diagonale est détectée grâce aux directions de la destination : NORTH-EST, NORTH-WEST, SOUTH-EAST, SOUTH-WEST.

- Relation en verticale : Pour placer la flèche de manière optimale entre les deux entités, on a calculé les coordonnées du point médian en termes de l'axe des ordonnées (y). Cela garantit que la flèche est centrée verticalement entre les deux entités, quelque soit leur position sur l'axe horizontal.
- Relation en horizontale : Pour placer la flèche de manière optimale entre les deux entités, on a calculé les coordonnées du point médian en termes de l'axe des abscisses (x). Cela garantit que la flèche est centrée horizontalement entre les deux entités, quel que soit leur position sur l'axe vertical.
- Relation en diagonale : Vous ajustez la position de la flèche en calculant les coordonnées d'un point central qui se trouve entre les deux entités, mais qui suit également la direction diagonale de la relation. Cela assure que la flèche est centrée et orientée correctement entre les entités.

Cette approche unifiée offre une visualisation épurée et une cohérence graphique, quel que soit l'agencement des éléments du diagramme. La documentation technique détaillée est disponible dans la classe **ARelationGraphic**, offrant une explication complète des logiques et des calculs sous-jacents à cette méthode.

IV.3.4 Positionnement manuel des relations

- **Ajout de points de contrôle** : pour rendre l'utilisation de l'application plus flexible, nous avons fourni la possibilité à l'utilisateur d'ajouter un point pour pouvoir ajuster les diagrammes manuellement. Cette

fonctionnalité est accessible via le menu `ctxMenu` du clic droit de la relation en choisissant l'option "Ajouter un point", une boîte de dialogue s'affichera pour confirmer le choix de l'utilisateur.

Pour ce faire, les coordonnées sont récupérées à travers un `EventHandler` et données en paramètre à la méthode `addPointInRelation` qui se charge de l'ajout du point à la liste des points, et qui dessine alors les lignes associées.

Cette fonctionnalité a été implémentée dans les classes `ArrowBody` (méthode `addPoint`), `ARelationGraphic` (méthode `addPointInRelation`), `RelationGraphicController` (`ctxMenu`) et `DialogAddPoint` dans le paquetage `ui.component.clazz.dialog`

- **Suppression de points de contrôle** : nous avons aussi intégré la possibilité de retirer un point des relations. Cette fonctionnalité est activée par un double-clic sur le point concerné, suivi d'une confirmation via une boîte de dialogue.

Pour implémenter cette fonctionnalité, un `MouseListener` a été ajouté à la classe `ArrowBody` dans la méthode `addMouseEventOnCircle` pour détecter les doubles clics sur les points.

Quand ce dernier est détecté, cela déclenche une notification aux observateurs de propriété, permettant ainsi aux autres classes concernées de réagir à cette suppression.

De plus, un `ChangeListener` a été instauré dans la classe `ARelationGraphic` pour surveiller et gérer les suppressions de points dans les relations graphiques.

Ces modifications ont été appliquées dans les classes `ArrowBody` (méthode `removePoint`), `ARelationGraphic` (ajout des événements et des `PropertyChange`) et `DialogRemovePoint` dans le paquetage `ui.component.clazz.dialog`

IV.3.5 Gestion des superpositions

Dans la version antérieure du code, l'affichage des relations présentait une inconstance notable, dépendant de l'ordre dans lequel les composants étaient créés ou modifiés. Il était observé que lorsqu'un utilisateur créait ou modifiait une relation après avoir ajouté une classe ou une note, ces relations se retrouvaient souvent superposées au-dessus des autres éléments, engendrant ainsi un affichage désordonné et peu intuitif du diagramme.

La solution apportée à ce problème consiste à revoir la manière dont les relations sont affichées à l'écran, indépendamment de l'ordre dans lequel elles sont créées ou modifiées. Ainsi, les relations sont désormais systématiquement placées en dessous des autres composants du diagramme, garantissant ainsi un affichage clair et cohérent.

Cette optimisation a été implémentée au sein de la classe `ARelationGraphic`, située dans le paquetage `ui.component.common.relation`. Plus précisément, la méthode `drawArrowBodyInGroup` a été modifiée afin de garantir que les relations soient toujours insérées au début du groupe, assurant ainsi qu'elles se retrouvent toujours en bas de la hiérarchie graphique.

IV.3.6 Exportation PDF

La gestion de l'exportation n'ayant pas été prise en compte dans la version antérieure, il était nécessaire de l'ajouter, car c'est une fonctionnalité importante. Pour ce faire, des changements ont été réalisés dans les trois fichiers :

- **pom.xml** : pour inclure la gestion de l'exportation vers des fichiers PDF, nous avons introduit la dépendance `pdfbox` le fichier `pom.xml`. Cette dépendance permet d'intégrer la bibliothèque `PDFBox`. En l'incorporant, nous offrons à l'application la possibilité d'interagir de manière étendue avec des documents PDF, ouvrant ainsi de nouvelles perspectives d'utilisation pour les utilisateurs.
- **MenuController** : le processus de création d'un document PDF à partir d'une capture d'écran implique plusieurs étapes. Tout d'abord, une capture d'écran est effectuée en utilisant un objet `SnapshotParameters` pour spécifier les paramètres de capture. Ensuite, l'image JavaFX capturée est convertie en un objet `BufferedImage` à l'aide de `SwingFXUtils.fromFXImage()`.

Les dimensions de la page PDF sont calculées en fonction de la taille de l'image capturée, en ajoutant un espace supplémentaire pour éviter que le contenu ne soit coupé lors de l'impression. L'utilisateur est invité à choisir l'emplacement et le nom du fichier PDF à enregistrer à l'aide d'un sélecteur de fichiers. Si un fichier est sélectionné, le document PDF est enregistré et les ressources sont libérées en fermant le document avec la méthode `doc.close()`.

- **Menu.fxml** : ajout d'un `MenuItem` dans la barre de menu pour l'export en PDF. Impliquant ainsi l'ajout de ce dernier par l'intermédiaire de `SceneBuilder`, dans le fichier `Menu.fxml`

V Difficultés rencontrées

Lors de la réalisation de ce projet, nous avons été confrontés à plusieurs défis significatifs, allant de la prise en main des outils de développement utilisés, à l'analyse des classes, jusqu'à la réalisation des objectifs. Les principaux ont été :

- **Mise à jour de l'environnement de développement** : l'adaptation aux divers environnements de développement, notamment JavaFX et Maven, a exigé une acquisition rapide de compétences et la mise à niveau des versions. Cette transition a demandé un investissement important en temps et en efforts pour assurer une intégration fluide.
- **Analyse du code** : l'analyse du code existant a également posé des difficultés. La densité du code, le grand nombre de classes et la documentation incomplète ont rendu cette tâche complexe. Comprendre le fonctionnement des fonctionnalités de l'application a exigé un temps considérable et une analyse approfondie.
- **Organisation et répartition des tâches** : en ce qui concerne l'organisation et la répartition des tâches, la complexité de l'analyse du code a influencé notre approche. Nous avons dû prendre le temps de diviser les tâches en fonction de leur nature et de leur niveau de difficulté, tout en tenant compte des compétences de chaque membre de l'équipe. Cette démarche nous a permis de surmonter les obstacles rencontrés et d'avancer de manière efficace malgré les défis.

VI Evolution potentielles

Dans la perspective d'amélioration continue de l'application UML Reverse, de nombreuses fonctionnalités peuvent être envisagées pour enrichir son potentiel. Les possibilités d'évolution sont multiples et peuvent être abordées de diverses manières, en fonction des besoins des utilisateurs et des avancées technologiques. Parmi les évolutions envisageables, nous identifions des améliorations notables qui pourraient significativement augmenter la valeur de l'application comme :

- Contournement automatique des relations : cette fonctionnalité permettrait aux relations de contourner automatiquement les obstacles, améliorant ainsi la clarté visuelle des diagrammes UML.
- Masquage des propriétés et méthodes : cette option pour cacher ou montrer sélectivement les propriétés et les méthodes des entités dans les diagrammes, permettrait une vue simplifiée pour une meilleure compréhension à divers niveaux d'abstraction.
- Gestion multi-projet : cette fonctionnalité offrirait la capacité de gérer et de travailler sur plusieurs projets simultanément.

Ces améliorations pourraient significativement renforcer l'efficacité, la flexibilité et la convivialité de l'application, la rendant plus adaptée aux besoins évolutifs de ses utilisateurs.

VII Conclusion

C'est avec un sentiment d'accomplissement que nous clôturons ce projet. Chacun de nous a porté sa pierre à l'édifice avec de la solidarité et de l'entraide, indispensables pour mener à bien notre projet. La force de notre collaboration a résidé dans notre capacité à tirer le meilleur de chacun et à surmonter les difficultés.

Ce projet nous a permis de mettre en application certains de nos acquis théoriques et nous familiariser avec de nouvelles technologies, nous poussant à affiner notre expertise et à élargir nos compétences techniques. Les délais nous ont appris à mieux gérer notre temps et les priorités.

Chaque étape nous a permis de d'améliorer et d'adapter notre méthodologie et de renforcer notre esprit d'équipe. Notre application de création de diagrammes de classe UML se distingue des solutions existantes sur le marché. En effet, nous avons réussi à répondre au besoin des étudiants et des enseignants en réalisant une solution innovante et gratuite.

Ce que nous retenons de cette riche expérience, le renforcement des liens entre les membres du groupe, l'acquisition de nouvelles compétences et connaissance approfondie des technologies.